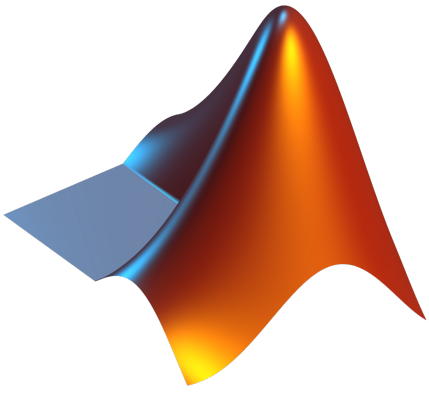
Matrices en MATLAB y Respuesta Transitoria

Fabrizio Benavidez[1]

[1] *Facultad de Ciencias Exactas, Físicas y Naturales, Ingeniería Aeronáutica*

Resumen. El presente trabajo aborda la implementación computacional de algoritmos numéricos y el análisis de sistemas de control dinámicos mediante MATLAB y Simulink. En una primera etapa, se desarrollan funciones iterativas utilizando bucles `for` y `while` para el cálculo de la sucesión de Fibonacci y la evaluación de límites numéricos en números primos. Posteriormente, se implementa el criterio de estabilidad de Routh-Hurwitz para la evaluación algebraica de polinomios característicos de cuarto orden. Finalmente, se estudia la respuesta en estado estacionario de un sistema de control retroalimentado sujeto a diferentes señales de entrada (escalón, rampa y parábola). Mediante la aplicación del Teorema del Valor Final y la variación de la función de transferencia de retroalimentación $H(s)$, se analiza el impacto del tipo de compensador sobre el error estacionario e_{ss} y la ubicación de los polos en lazo cerrado para determinar la estabilidad del sistema.

Palabras claves: MATLAB, Simulink, ciclos, estabilidad.



22 de septiembre de 2026

Semana 5

[1] `fabrizio.benavidez@mi.unc.edu.ar`

Guía 09

La sucesión de Fibonacci se define como:

$$f_n = f_{n-1} + f_{n-2} \tag{1}$$

donde $f_0 = 0$ y $f_1 = 1$. Por lo tanto, para el siguiente valor de f es:

$$f_2 = 0 + 1 = 1$$

Para f_3

$$f_3 = f_2 + f_1 = 1 + 1 = 2$$

y así sucesivamente.

El siguiente código de MATLAB representa la función de Fibonacci para cualquier n.

```
    function fn = fibonacci2(n)
        a = 0; b = 1;
        arr = 1:1:n;
        for i = arr
            fn = a + b;
            temp = a;
            a = fn ; b = temp;
        end
    end

```

Código 1. La función `fibonacci2` calcula el valor de la sucesión para un valor n

Si llamamos la función y le ponemos el valor de $n = 40$, el valor obtenido es 102334155. Los siguientes códigos representan lo mismo pero usando bucles `while`.

```
function fn = fibonacci(n)
    a = 0 ; b = 1; %f_0 = 0,  f_1 = 1
    m = n - 1; %le resto 1 ya que arranco con 2 valores de la
    sucesi n
    i = 0;
    while i <= m
        fn = a + b;
        i = i + 1;
        temp = a;
        a = fn; b = temp;
    end
end


function fn = fibonacci_10000()
    a = 0 ; b = 1;
    fn = a + b;
    while fn < 10000
        fn = a + b;
        temp = a;
        a = fn ; b = temp;
    end
end


function fn = fibonacci_v(v)
    a = 0; b = 1;
    i = 1;
```

```
28      fn = 0;
29      while fn < v
30          fn = a + b;
31          i = i + 1;
32          temp = a;
33          a = fn ; b = temp;
34      end
35  end
```

Código 2: La función `fibonacci` representa la función para calcular los primeros n valores de la sucesión. `fibonacci_10000` calcula el valor de la sucesión una vez que es mayor a 10000. Por último, `fibonacci_v` le pasamos como argumento un valor v donde calculará el último valor de la sucesión cuando $f_n > v$.

Criterio de estabilidad de Routh-Hurwitz

El criterio de estabilidad de Routh-Hurwitz dice si existen o no raíces inestables en una ecuación polinomial, sin tener que obtenerlas en realidad. Este criterio de estabilidad sólo se aplica a los polinomios con una cantidad finita de términos.
El procedimiento en el criterio de estabilidad de Routh es el siguiente:

1. Se escribe el denominador de la función transferencia de la forma siguiente:

$$a_0 s^n + a_1 s^{n-1} + \cdots + a_{n-1} s + a_n = 0 \tag{2}$$

   donde los coeficientes son cantidad reales. Se supone que $a_n \neq 0$; es decir, se elimina cualquier raíz cero.

2. Si todos los coeficientes son positivos, se ordenan los coeficientes del polinomio en filas y columnas con el patrón siguiente:

$$\begin{array}{rllll} s^n & a_0 & a_2 & a_4 & a_6 \cdots \\ s^{n-1} & a_1 & a_3 & a_5 & a_7 \cdots \\ s^{n-2} & b_1 & b_2 & b_3 & b_4 \cdots \\ s^{n-3} & c_1 & c_2 & c_3 & c_4 \cdots \\ . & . & & & \\ . & . & & & \\ . & . & & & \\ s^2 & e_1 & e_2 & & \\ s^1 & f_1 & & & \\ s^0 & g_1 & & & \end{array}$$

   El proceso de armar los valores de b, $c \cdots$, e, f y g se realiza mediante un proceso parecido a tomar el determinante de una matriz 2×2.

El criterio asegura que si existe algún cambio de signo en la primera columna de los coeficientes, entonces el sistema es inestable (tiene una raíz en el semiplano derecho).

El siguiente ejemplo muestra cómo aplicar el criterio para un sistema de 4to orden:

$$y^{(4)} + 2\dddot{y} + 4\ddot{y} + \dot{y} + 1 = r(t) \tag{3}$$

La matriz H es el arreglo de coeficientes definido anteriormente:

$$H = \begin{bmatrix} 1 & 4 & 1 \\ 2 & 1 & 0 \\ \alpha_1 & \alpha_2 & 0 \\ \beta_1 & 0 & 0 \\ \gamma_1 & 0 & 0 \end{bmatrix}$$

donde

$$\alpha_1 = 3{,}5, \qquad \alpha_2 = 1, \qquad \beta_1 = 0{,}4286, \qquad \beta_2 = 0, \qquad \gamma_1 = 3{,}5$$

El siguiente código representa la función que evalúa si el sistema es o no estable:

```
function estable(M)
    first_row = M(1);
    for i = 1:length(first_row)
        if i < 0
            disp('Sistema no estable!')
            break
        end
    disp('Sistema estable!')
    end
end

function RH_4(y)
    len = length(y);
    H = zeros(len, 3);
    a4 = y(1); a3 = y(2); a2 = y(3); a1 = y(4); a0 = y(5);
    alpha1 = ((a3*a2) - (a4*a1))/a3;
    alpha2 = ((a3*a0) - (a4*0))/a3;
    beta1 = ((alpha1*a1)-(a3*alpha2))/alpha1;
    gamma1 = ((beta1*alpha1)-(0*alpha1))/beta1;

    H(1, [1:3]) = [a4, a2, a0];
    H(2, [1:2]) = [a3, a1];
    H(3, [1:2]) = [alpha1, alpha2];
    H(4, 1) = beta1;
    H(5, 1) = gamma1;

    estable(H);
    H
end

coef1 = [1, 2, 4, 1, 1];
% coef2 = [3, 2, 2, 0, -1];
RH_4(coef1)
```

Código 3: Criterio de Routh aplicado al sistema de 4to orden

Por último, veamos el siguiente problema. Si p es un número primo se puede obtener un nuevo número primo q de la siguiente manera:

$$q = 2 \cdot 3 \cdot 5 \cdot 7 \cdots p + 1 \tag{4}$$

El siguiente código implementa un bucle para buscar el último número primo que puede manejar MATLAB, el cual es: $2{,}811 \times 10^{307}$.

```
for i = 300:900
    q = prod(primes(i)) + 1;
    if ~isfinite(q)
            q = prod(primes(i - 1) + 1);
            break
```

```
6       end
7   end
```

Guía 10

Veamos la siguiente figura

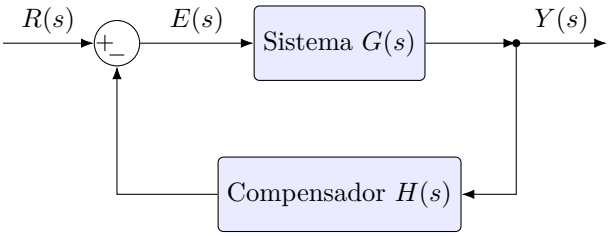


Figura 1. Diagrama de bloques de un sistema de control retroalimentado sin controlador.

Dicha figura representa un sistema de control retroalimentado mediante $H(s)$. En la salida del punto suma se representa el error que surge entre la entrada y la salida multiplicada por el compensador. El error es entonces

$$E(s) = R(s) - H(s)Y(s)$$

Si dividimos el error por la entrada, se obtiene

$$\frac{E(s)}{R(s)} = 1 - \frac{H(s)Y(s)}{R(s)}$$

Pero $Y(s)/R(s)$ es la función de transferencia en lazo cerrado, por lo tanto

$$\frac{E(s)}{R(s)} = 1 - H(s)\left(\frac{G(s)}{1 + H(s)G(s)}\right) = \frac{1}{1 + G(s)H(s)}$$

El teorema del valor final ofrece una forma conveniente de determinar el comportamiento en estado estacionario de un sistema estable. Como $E(s)$ es

$$E(s) = \frac{1}{1 + G(s)H(s)}R(s)$$

el error en estado estacionario es

$$e_{ss} = \lim_{t\to\infty} e(t) = \lim_{s\to 0} sE(s) = \lim_{s\to 0} \frac{s}{1 + G(s)H(s)}R(s) \tag{5}$$

La figura 2 representa el sistema, donde los parámetros son:

$$a = 1, \qquad b = 2, \qquad c = 1, \qquad d = 10, \qquad e = 5, \qquad f = 8$$

La siguientes figuras representan el error para los 3 tipos de entradas establecidos anteriormente

Si evaluamos el límite 5 para una entrada escalón, obtenemos:

$$e_{ss} = e_p = \frac{1}{\frac{b}{f} + 1} = \frac{f}{b + f} = \frac{4}{5} = 0{,}8 \tag{6}$$

el cual quiere decir que el error según entrada escalón es del 80 % (Observece la figura 3 para la entrada escalón).

Ahora bien, cambiemos la retroalimentación (compensador) por la siguiente función transferencia: $1/s$. Ahora si evaluamos el límite para la entrada escalón, se obtiene el resultado de 0, es decir que no hay error en la entrada escalón para el sistema. Ahora, si la entrada es una rampa

$$e_{ss} = e_v = \frac{f}{b} = 4 \tag{7}$$

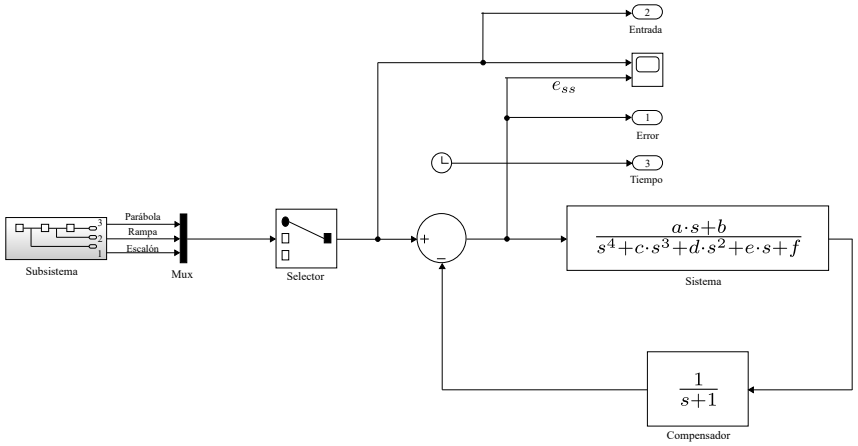


Figura 2. Sistema de control retroalimentado. El subsistema representa 3 entradas: escalón, rampa y parábola. Se utiliza un *Selector* para elegir de forma dinámica el tipo de entrada que quiero recibir en el sistema.

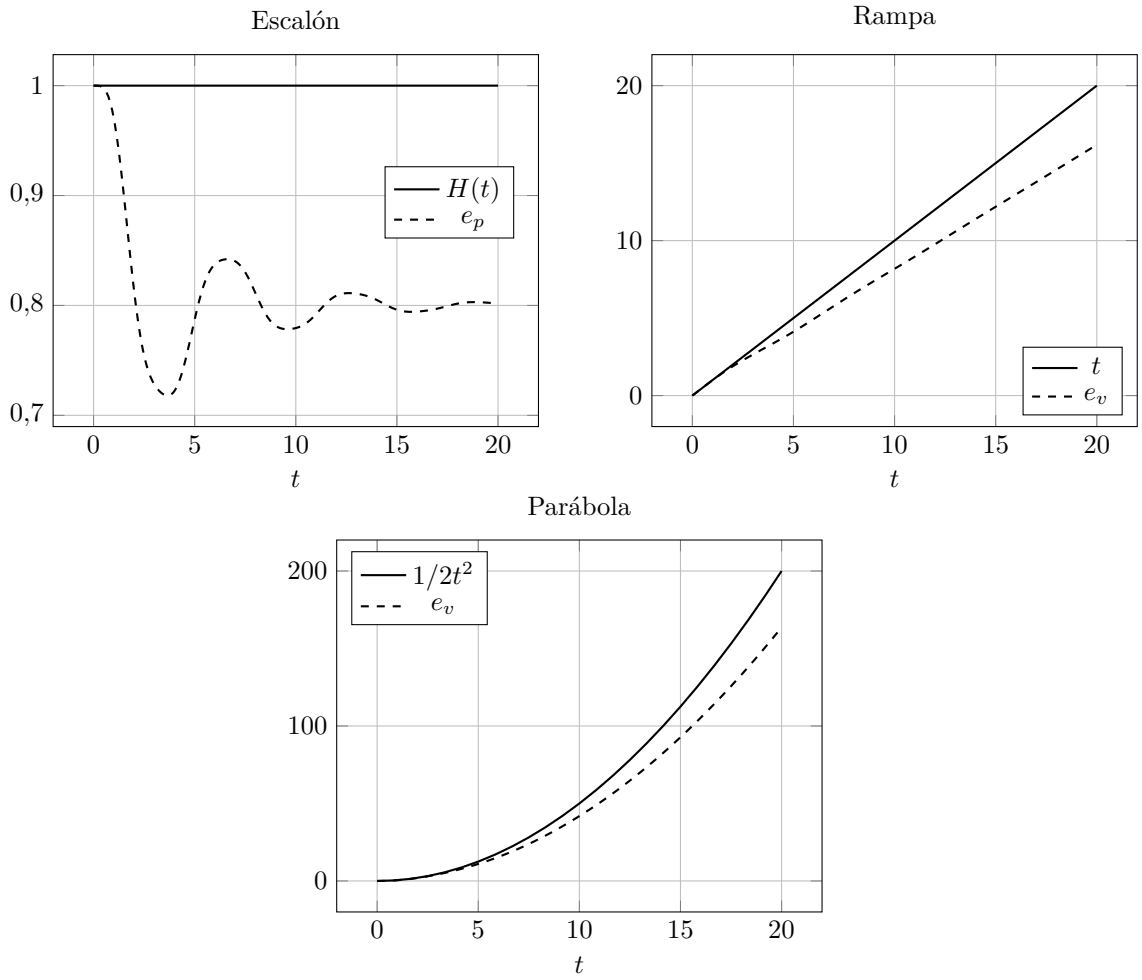


Figura 3. Errores del sistema asociados a distintas entradas. Se observa que en cada una de ellas existe una diferencia entre la entrada y el error del sistema.

Es decir, el error de velocidad es del 400 %. A diferencia del compensador analizado previamente, el error velocidad divergía.

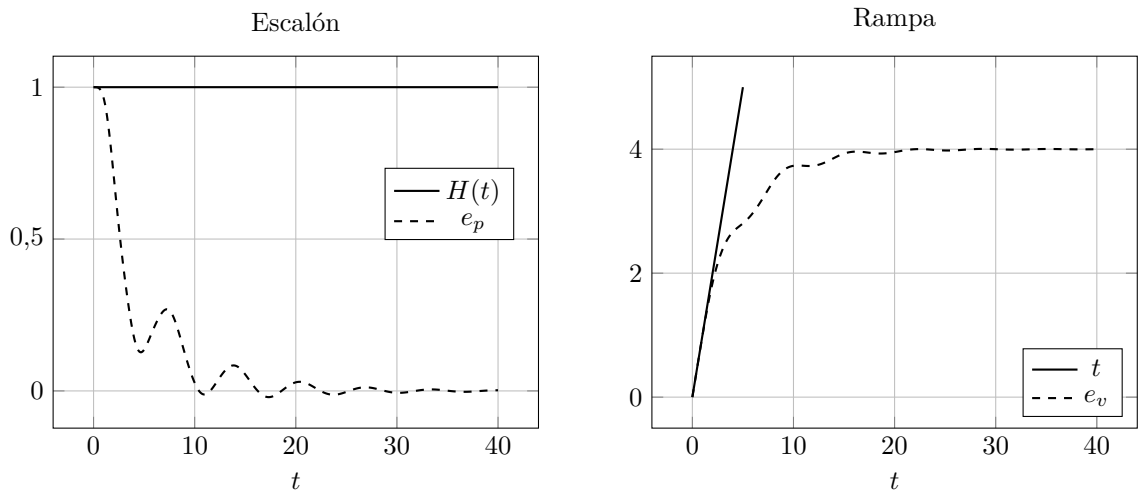


Figura 4. Errores del sistema asociados a distintas entradas pero con un integrador como compensador. Para la entrada escalón el error tiende a 0 pero para la entrada rampa el error converge a un límite (anteriormente divergía). Para la entrada parabólica sucede parecido al caso analizado anteriormente

Veamos el último caso: $H(s) = 1/s^2$. Para comprender qué pasa en este caso, podemos acudir a las funciones `tf`, `feedback` y `pole`. El siguiente código representa el cálculo de los polos para el sistema.

```
a = 1;
b = 2;
c = 1;
d = 10;
e = 5;
f = 8;

Gs = tf([a, b], [1, c, d, e, f]);
Hs = [tf([1], [1, 1]), tf([1], [1, 0]), tf([1], [1, 0, 0])];
Fs = feedback(Gs, Hs(3)); %funci n transferencia de lazo cerrado
polos_fs = pole(Fs);
```

Código 4: Cálculo de los polos al sistema

Por lo tanto, para el sistema con compensador $1/s^2$ se obtienen los siguientes polos.

$$p_{1,2} = -0{,}2484 \pm 2{,}9696i$$
$$p_{3,4} = -0{,}3506 \pm 0{,}7433i$$
$$p_{5,6} = 0{,}0991 \pm 0{,}5689i$$

Es decir, los polos presentan parte real positiva y por lo tanto el sistema es inestable.